HRBP面（四面）

非技术面试准备材料 · 答题模板

1. 遇到的最大困难是什么？如何解决？

我在 AI4Test 项目中遇到的最大挑战，是如何把测试人员依赖领域经验完成的工作，转化成 Agent 能够理解、执行和验证的任务流程。

因为我本身不是传统的测试业务人员，刚开始对一些具体测试场景的理解不够深入。例如，拿到一个测试需求后，应该关注哪些测试点、具体执行哪些步骤、出现异常后应该先检查什么，以及如何逐步缩小范围定位根因。这些内容并不是简单调用大模型或者检索几份文档就能解决的。
针对这个问题，我主要从三个方面推进。

第一，主动补齐业务知识。我不会只根据自己的理解设计 Agent，而是结合实际需求和测试文档梳理业务流程，并与熟悉相关测试场景的同事沟通。对于不清楚的地方，我会把问题具体化，例如需求对应哪些测试点、每个测试点需要什么前置条件、执行结果如何判断，以及失败后通常会检查哪些指标和日志，再针对这些问题向业务同事请教。

第二，把业务经验沉淀成系统能力。对于沟通得到的业务规则和处理经验，我会进一步整理成结构化的任务步骤、输入输出约束、工具调用要求和结果校验规则，融入相应的 Agent 或 Skill 中。这样就不是每次都依赖模型临时推理，而是让 Agent 在明确的业务边界内完成任务。

第三，建立反馈和迭代机制。Agent 的结果不能只看回答是否合理，还要看测试步骤是否可执行、关键测试点是否遗漏、工具调用是否正确，以及故障分析有没有证据支撑。如果发现结果不符合预期，我会结合执行记录和业务同事的反馈分析问题究竟来自业务理解、知识检索、任务规划还是工具执行，再针对具体环节优化。

这件事让我认识到，工业场景中的 Agent 开发并不是单纯的模型应用开发。开发人员需要主动理解业务、和领域专家建立有效协作，再把专家经验沉淀为可复用、可验证的系统能力。

2. 你具体怎么拉人、怎么和测试同事沟通？

我拉人主要分三步：先找对关键人，再降低参与成本，最后用结果建立信任。

第一，找对关键人。我不会一开始就拉一大群开会，而是先找测试负责人和一两位资深测试同事，跟他们讲清楚三件事：这个 Agent 要解决什么问题、能帮他们省什么、需要他们投入多少时间。先争取关键人的支持，再由他们帮我推荐合适的同事，比我自己挨个找人有效得多。

第二，降低参与成本。测试同事本身工作很忙，所以我尽量不让他们从零开始写。我会先把需求文档、历史用例和流程整理成一版草稿，带着具体问题去沟通，比如"这个场景的前置条件是不是 A""失败后一般先看哪几个日志"，而不是问"这个业务怎么测"这种大而空的问题。形式上尽量轻量：日常用群和文档异步确认，每周固定半小时做一次评审，有争议的点再单独拉会。

第三，形成固定的协作机制。我们约定了一个简单流程：我出 Agent 草稿，测试同事只做 review 和纠错，确认后的结论由我整理进知识库或规则里；每个阶段选一两个典型场景做试点，把节省的时间、发现的问题量化出来给大家看。这样他们能感受到自己是在"定义标准"，而不是"给开发打杂"，参与意愿会高很多。

沟通时我会注意两点：一是用他们的语言，尽量用测试用例、缺陷、复现步骤这些他们熟悉的表达；二是每次沟通结束前复

述一遍我的理解，请他们确认，避免我自以为懂了。遇到同事担心 Agent 替代人工，我会明确说清楚定位是辅助和提效，关键判断仍然由测试同事把关，并且把他们的经验署名沉淀下来。

3. 测试流程不懂，怎么知道 Agent 生成的测试步骤对不对?

我不会用"模型自己说对不对"来判断，而是用三层验证。

第一层是业务标准。我会先和测试同事把"什么样的步骤算对"定义清楚，包括前置条件是否完整、操作步骤是否可执行、预期结果是否明确、关键测试点有没有覆盖。然后把历史真实需求和缺陷整理成一套评测集，由测试同事标注出正确步骤和必须覆盖的测试点，作为回归验证的基准。没有这套标准，后面所有讨论都会变成主观争论。

第二层是可执行性。Agent 生成的步骤不能只停留在文字上，要在测试环境里真正执行，看能不能跑通、断言能不能验证、产生的日志和数据是否符合预期。对于不能自动执行的部分，我会把它拆成人工可操作的检查清单，请测试同事按清单实际走一遍。

第三层是结果可追溯。Agent 给出的每个测试点、每个判断，我都会要求能对应到具体的需求条目、文档片段或历史用例，并保留执行记录和工具调用日志。这样测试同事 review 时不是在"信或不信"，而是在核对证据。如果他们改动了某一步，我会记录修改原因，把高频问题沉淀成规则或示例，再进入下一轮评测。

衡量上我会看几个指标：步骤可执行率、关键测试点覆盖率、人工修改率，以及漏测和误报的情况。先用小范围试点、人工把关，稳定后再逐步扩大范围。

4. 出了问题，你怎么和业务同事一起找根因?

出问题后我会按"先止损、再复现、后定位"的顺序推进，并且一定拉上业务同事一起看。

第一步是先控制影响。确认问题影响哪些场景、是否已经产生错误结果，必要时先回退到上一个稳定版本或切回人工流程，同时把现场保留下来：输入数据、Agent 的完整执行轨迹、工具调用参数和返回、使用的模型和 Prompt 版本、相关日志。

第二步是一起复现。我会把问题收敛成最小可复现的输入，约上发现问题的测试同事一起看，避免我一个人在那边猜。分工上通常是他判断业务上应该是什么结果，我负责查链路上哪一步偏了。

第三步是分层定位。我会沿着"需求理解 → 知识检索 → 任务规划 → 工具调用 → 执行环境 → 结果校验"逐层排除：是业务规则没有沉淀进去，还是检索到了错误的文档，还是规划漏了步骤，还是工具参数传错、环境本身有问题。每一层都用证据说话，而不是先入为主地认为是大模型不行。

定位清楚后，我会把结论、修复方案、责任人和时间点写进问题记录，修复后做回归，并把这次 case 补进评测集，避免同类问题再出现。沟通上我的原则是先把问题认下来、说清楚影响和临时方案，再讨论原因，不甩锅也不过度承诺。

5. 你最大的困难到底是技术问题，还是业务问题?

如果只选一个，我认为主要矛盾是业务问题，技术是实现手段。

一开始我也以为难点在技术上，比如模型效果、工具调用稳定性。但做的过程中发现，模型能力本身往往够用，真正卡住的是业务侧：什么算测试点覆盖完整、什么算步骤正确、失败了应该先看哪里，这些标准在业务同事脑子里，是隐性的，文档里也不完整。如果这些定义不清楚，技术做得再好也只是在优化一个错误的目标。

当然技术问题也存在，比如结果不稳定、评测不好做、工具调用会出错，但这些可以通过工程手段逐步收敛：加约束、加校验、做评测集、人工兜底。而业务知识的获取、和领域专家达成共识、把经验变成可执行的规则，成本更高，也更难被替代。

所以我的判断是：业务问题是主要矛盾，技术问题是可以持续迭代的次要矛盾。这也影响了我的工作方式——我会先花时间把验收标准和业务规则问清楚，再动手做技术方案，而不是上来就调模型。这件事也让我从一个只关注技术实现的人，转变成愿意先理解业务、再用工程手段解决问题的角色。

常见 HRBP 面试题与参考答案

1. 请做一个简单的自我介绍

我是一名偏 AI 应用方向的开发工程师，主要做 Agent 相关的能力建设，最近一段时间的核心工作是把大模型能力落到实际业务场景里。

在项目里我负责过 Agent 的整体方案设计和落地，包括任务规划、工具调用、知识检索、结果校验，以及配套的评测和反馈机制。我比较关注的不只是"能不能跑通"，而是结果是否稳定、可验证、能持续迭代。

在推进过程中，我需要经常和业务、测试同事协作，把他们的经验整理成 Agent 能执行的规则和流程，这也让我比较习惯从业务目标出发去设计方案，而不是只从技术角度出发。

我选择这个机会，是因为【贵司的业务方向 / 这个岗位的职责】和我想长期投入的方向比较一致，我也希望在更完整的业务场景里继续把 Agent 做深。

2. 为什么想换工作?

主要有两个原因，都是偏发展性的，不是和团队或领导有矛盾。

第一是业务场景的深度。我目前在做的 Agent 项目已经走过了从 0 到 1 的阶段，基础链路和协作机制相对稳定了，我希望进入一个业务更复杂、对可靠性要求更高的场景，把 Agent 的能力做得更深，而不是停留在重复建设上。

第二是成长空间。我更希望在一个重视工程质量和长期积累的团队里，把评测、可观测性、效果迭代这套东西做扎实。这也是我选择贵司的原因之一。

我在上一段经历里和同事、领导合作都比较顺畅，离开是个人职业规划的选择，也会把手上的工作交接清楚。

3. 为什么选择我们公司 / 这个岗位?

我主要看三点。

第一是方向匹配。贵司在【业务方向】上的投入和我的技术积累比较一致，我做过的 Agent 规划、工具调用、效果评测这些能力，能够比较快地用上。

第二是岗位的成长性。这个岗位不只是做单点功能，而是需要把 AI 能力和真实业务结合起来，这正好是我过去一年在解决的问题，也是我想继续做深的方向。

第三是团队和工程氛围。我在面试过程中感受到团队比较重视实际效果和工程落地，而不是只追求概念，这和我做事的习惯比较一致。

所以对我来说，这不只是一次换工作，而是希望在合适的场景里把同一件事做得更专业。

4. 你未来 3 到 5 年的职业规划是什么?

我希望在 3 到 5 年内，成为能够独立负责一个 AI 应用方向的技术骨干。

第一阶段，也就是入职后的一年左右，我会先把业务和系统吃透，能独立承担模块的设计和交付，把当前岗位的事情做到可靠。

第二阶段，两到三年内，我希望在 Agent 的某个方向形成比较完整的工程方法论，比如任务规划、工具生态、效果评测和线上稳定性，能够从需求定义到上线迭代端到端负责。

更长一点，我希望能在团队里承担一定的技术方案设计和带人工作，把自己的经验沉淀成团队可复用的规范和工具。

整体上我的规划是往"技术深度 + 业务理解"的方向走，而不是单纯追求管理岗位。

5. 你的优点和缺点分别是什么?

优点方面，我认为比较突出的是两点。

一是遇到不熟悉的领域愿意先补认知。比如做测试 Agent 时我本身不是测试出身，我会主动拆解业务问题、找领域同事确认，而不是凭感觉做方案。

二是比较重视结果的可验证性。我做东西习惯先想清楚怎么衡量对不对，会主动做评测集、留执行记录，尽量不让方案停留在"看起来可以"。

缺点方面，我有时候会过于关注细节，在一个问题上钻得比较深，导致节奏偏慢。后来我会在开始前先明确这件事的目标和时间盒，区分哪些必须做扎实、哪些可以先出可用版本，再逐步迭代。另一个是我在公开表达上还不算特别强，这两年也在通过多做分享和评审来刻意练习。

6. 你的领导和同事一般怎么评价你?

综合他们的反馈，比较一致的评价是：做事比较踏实，遇到问题愿意往下查，交付的东西比较让人放心。

领导通常会说我"主动性比较强"，遇到不清楚的需求会主动去对齐，而不是等着被安排。同事更多会提到我"好沟通"，因为我习惯把问题具体化，也会把结论沉淀成文档，减少来回确认的成本。

当然也有反馈说我有时候想得太多、推进节奏可以更快一些，这一点我自己也认同，现在会有意识地先定目标和时间点，避免在细节上过度投入。

7. 你最有成就感的一件事是什么?

最有成就感的是把 Agent 从一个 demo 做到能在真实业务里被同事持续使用。

一开始它只能回答一些通用问题，业务同事的反馈是"看起来可以，但不敢用"。后来我做了几件事：把业务规则沉淀成结构化流程，给关键结论加上证据来源，建立一套小规模的评测集，并且让测试同事参与 review。经过几轮迭代后，它生成的测试步骤开始能被直接采纳，人工修改率明显下降。

让我最有成就感的不是技术指标本身，而是业务同事开始主动来提需求，说明这个东西真的进入了他们的工作流，而不是一个演示项目。这也让我确认了自己做 AI 应用的方向是对的：价值最终要体现在业务愿不愿意用。

8. 讲一次失败或受挫的经历

有一次我负责的一个 Agent 功能，前期 demo 效果很好，但上线试点后业务同事的使用率很低。

我一开始以为是模型效果不够，后来去访谈才发现，真正的问题是它没有嵌入同事现有的工作流程：他们需要额外打开一个页面、手动复制输入，而且结果格式和他们习惯的用例模板不一致。也就是说，我优化的是自己认为重要的指标，而不是用户的实际路径。

这次之后我调整了做法：先跟使用方确认工作流和验收标准，再决定技术方案；能嵌进现有工具就不另起入口；每个版本都找真实用户试用，而不是只看离线评测。

这件事对我的影响是，我不再默认"技术效果好就等于有价值"，而是会先问清楚谁用、在什么场景用、怎么判断有用。

9. 和同事意见不一致或者发生冲突时，你怎么处理?

我一般会先把"观点冲突"和"情绪冲突"分开处理。

如果是方案分歧，我会先回到共同目标：我们各自想解决什么问题、判断标准是什么。然后把分歧拆成具体可验证的点，比如用数据、试点结果或者小范围验证来判断，而不是靠谁声音大。如果一时验证不了，我会尊重负责人的决策，先把事情推进，同时把风险和我的保留意见记录清楚。

如果是情绪上的冲突，我会先私下沟通，避免在群里激化。先听对方把话说完，确认他真正在意的是什么，再表达我的考虑。大多数情况下，冲突背后是信息不对称或者目标不一致，把这两点对齐后就好解决了。

我的原则是对事不对人，不翻旧账，也不为了表面和气回避问题。

10. 你怎么看待加班和压力?

我认为加班应该是为了解决问题，而不是常态。

如果是项目关键节点、线上问题或者紧急交付，我愿意配合，也会主动承担。但我更倾向于通过提前规划、拆解任务和暴露风险来减少无效加班，比如把不确定的事情尽早验证，避免最后集中爆发。

压力方面，我一般会先分清压力来自哪里：是任务不清晰、能力不足，还是时间不够。任务不清晰就去对齐，能力不足就去

补，时间不够就排优先级并及时同步。把事情拆小、写下来，压力会小很多。我也比较注意长期节奏，不会靠持续熬夜来换短期产出。

11. 你的期望薪资是多少?

我目前的期望是【期望区间】。这个数字主要基于我当前的能力和职责，也参考了市场上同类岗位的水平。

不过薪资不是我最看重的因素，我更关注岗位的成长空间和业务方向。如果整体机会合适，具体结构可以在 `offer` 阶段再沟通。我也想了解一下贵司这个岗位的薪资范围和构成，方便我们对齐预期。

12. 你目前还有其他 offer 吗?

目前有一些在流程中，但我还没有做最终决定。

我更看重的是岗位方向和团队，而不是单纯比较 `offer` 数量。贵司这边是我比较认真在考虑的选项，所以也想借这次沟通把业务规划、团队分工和成长路径了解清楚，再综合判断。

13. 你平时怎么学习新技术?

我一般分三步：先建立整体认知，再做小项目验证，最后沉淀成可复用的东西。

第一步是看官方文档、源码或者高质量的技术分享，先把概念和边界搞清楚，避免只记住结论。

第二步是动手做一个最小可运行的 `demo`，把关键链路跑通。比如学 `Agent` 相关能力时，我会自己实现一遍任务规划、工具调用和结果校验，遇到问题再回头查资料。

第三步是总结成文档或者内部分享，把踩过的坑和适用边界记下来。我觉得能讲清楚、能被别人复用，才算真正学会。

14. 在团队里你通常扮演什么角色?

我通常是比较偏"推进和落地"的角色。

一方面我会主动把模糊的需求拆成可执行的任务，明确输入输出和验收标准；另一方面我会关注跨角色协作，把业务、测试和开发之间的信息对齐，减少来回返工。

在技术讨论里我不一定是声音最大的那个，但比较习惯把方案的风险和边界讲清楚，也愿意承担那些需要深入排查的问题。如果团队需要，我也不排斥做文档、评测规范这类"不显眼但重要"的基础工作。

15. 如果领导安排了一件你不认同的任务，你会怎么做?

我会先确认自己是不是掌握了完整信息。很多不认同是因为我只看到了技术侧，而领导可能掌握业务优先级或者时间窗口。

确认之后，如果仍然有不同意见，我会私下把顾虑讲清楚，重点说风险和替代方案，而不是只说"我不同意"。比如这个方案可能带来的维护成本、对现有系统的影响，以及我建议的做法。

表达完之后，如果决策还是不变，我会按决定执行，并把自己担心的风险点记录下来、提前做兜底。我认为职业性体现在：可以充分讨论，但决定之后不阳奉阴违。

16. 入职后你打算怎么快速上手?

我会分三步走。

第一个月，重点是搞清楚业务和系统：业务目标是什么、核心流程怎么走、系统边界和关键模块在哪里，同时把代码和文档过一遍，遇到不清楚的就找对应同事确认。我会给自己列一个问题清单，尽快把认知盲区补上。

第二到第三个月，争取独立承担一个完整的模块或需求，从方案设计到上线走一遍，在做的过程中验证自己的理解，并建立和上下游同事的协作方式。

三个月之后，我希望能在负责的范围内做持续优化，比如效果评测、稳定性或者效率提升，并逐步把经验沉淀成文档。整体原则是先融入、再交付、后优化，不急于证明自己。

17. 你还有什么问题想问我们?

我有三个问题想了解一下。

第一，这个岗位未来半年最重要的目标是什么，团队目前最希望解决的是哪类问题？这样我能判断自己的能力和岗位重点是否匹配。

第二，团队目前的分工和协作方式是怎样的，这个岗位会更多地独立负责，还是和业务、算法、工程同学紧密配合？

第三，团队怎么看这个方向未来一两年的发展，以及对这个岗位的成长预期是什么？

至于薪资福利和流程，我了解得差不多了，如果有需要可以后续再沟通。